Architectural Trade-offs in Mobile Runtime Environments: A Comprehensive Analysis of JIT and AOT Compilation

1. Theoretical Foundations of Execution Models

The execution of software on mobile devices is governed by a fundamental choice in how high-level source code is translated into machine-readable instructions. This translation process is not merely a technical detail but a definitive architectural decision that dictates the performance, energy efficiency, and security profile of the mobile operating system. As mobile hardware has evolved from constrained embedded systems to powerful multi-core computing platforms, the strategies for code execution have shifted dynamically along a spectrum ranging from pure interpretation to static, ahead-of-time compilation.

1.1 The Compilation Spectrum and Von Neumann Constraints

At the heart of the execution debate lies the constraints of the Von Neumann architecture, where instructions and data share a common memory space, and the central processing unit (CPU) executes instructions sequentially. The efficiency with which a runtime environment feeds this pipeline determines the user experience.

1.1.1 Interpretation: The Fetch-Decode-Execute Loop

At the most basic end of the spectrum lies interpretation. In a pure interpreter model, the runtime environment reads the source code or an intermediate bytecode representation instruction by instruction. For every instruction encountered, the interpreter executes a corresponding routine in the host machine's native language. This process constitutes a software-based "fetch-decode-execute" loop that occurs entirely at runtime.

The primary advantage of interpretation is its simplicity and immediacy. There is zero compilation latency; the program begins execution the moment the interpreter is invoked. Furthermore, interpretation offers high portability. The interpreter itself must be compiled for the target architecture, but the source code or bytecode it executes remains platform-agnostic.[1] This allows the same application package to run on ARMv7, ARM64, and x86 architectures without modification. However, the performance cost is severe. The overhead of the dispatch loop—fetching the instruction, switching on the opcode, and jumping to the implementation—often dwarfs the actual work performed by the instruction itself. Modern interpreters typically use "threaded code" (direct threading) to reduce this dispatch overhead, where the address of the next implementation is stored at the end of the current one, but the branch prediction penalties on modern pipelined CPUs remain

significant. In mobile contexts, where CPU cycles are directly correlated with battery drain, pure interpretation is generally considered too inefficient for compute-intensive logic, though it remains useful for non-critical paths or highly dynamic scripting scenarios.

1.1.2 Ahead-of-Time (AOT) Compilation

At the opposite end of the spectrum is Ahead-of-Time (AOT) compilation. In this model, the translation from source code (or bytecode) to native machine code occurs before the program is ever executed—typically during the build process on a developer's machine or during installation on the target device.[2] The compiler performs a comprehensive static analysis of the codebase, applying aggressive optimizations such as dead code elimination, loop unrolling, and register allocation, eventually producing a standalone binary executable.
The AOT model excels in peak throughput and startup performance. Since the code is already in native form, the device incurs no runtime translation cost. The application launches immediately, limited only by I/O and initialization logic.[1] Furthermore, because the heavy lifting of optimization is done beforehand, the runtime memory footprint is reduced; there is no need to load a compiler or a profiler into the device's RAM. However, AOT suffers from rigidity. It cannot adapt to the runtime environment. Static analysis must be conservative; for instance, if a method is virtual, the static compiler often cannot determine the precise target of the call and must generate a slower dynamic dispatch instruction, whereas a dynamic compiler might see that only one implementation exists in practice and inline it.

1.1.3 Just-In-Time (JIT) Compilation

Just-In-Time (JIT) compilation represents a hybrid approach that attempts to combine the portability of bytecode with the performance of native code. In a JIT environment, the code is distributed as platform-independent bytecode. When the application runs, the Virtual Machine (VM) loads this bytecode. Initially, it may interpret the code to ensure fast startup. However, as execution proceeds, the VM monitors the execution frequency of methods and loops. When a specific block of code is identified as "hot"—that is, frequently executed—the JIT compiler is triggered.
The JIT compiler translates this hot bytecode into native machine code *during* execution.[3] This allows for adaptive optimization. The JIT compiler has access to runtime profile data that an AOT compiler lacks. It knows exactly which branches are taken, which types are passed to polymorphic methods, and which loops are active. Using this data, it can perform aggressive Speculative Optimizations.[4] For example, if a variable has always been an integer during the first thousand iterations of a loop, the JIT can generate highly optimized machine code that assumes it will remain an integer, guarding this assumption with a cheap check. If the check fails, the system performs a "deoptimization," reverting to the interpreter.

1.2 The "Iron Triangle" of Compilation

In the design of mobile runtimes, architects navigate a set of conflicting constraints often referred to as the "Iron Triangle" of compilation. While the classic project management triangle balances scope, cost, and time [6], the compilation triangle balances **Startup Latency**,

Peak Throughput, and **Memory Footprint**.[1] This triangulation is critical because improving one metric often degrades the others.

Vertex	Definition	Trade-off Implication
Startup Latency	The time required from the user tapping an icon to the app becoming responsive.	AOT offers the best startup latency because no compilation occurs at runtime. JIT suffers here because it must either interpret slowly or spend time compiling before code runs at speed.[2] Empirical data suggests AOT startup is 40-70% faster than JIT.[2]
Peak Throughput	The maximum raw execution speed of the application once it has reached a steady state.	JIT theoretically offers the highest peak throughput due to Profile-Guided Optimization (PGO) based on actual runtime data, enabling optimizations (like aggressive inlining) that static AOT cannot safely perform.[2]
Memory Footprint	The amount of RAM consumed by the application and the runtime environment.	JIT has the highest memory cost. It requires memory for the bytecode, the generated native code, the profiling data, and the compiler itself.[9] AOT is leaner, requiring only the native code, but its binary size on disk (Storage) is larger.

Mobile devices are uniquely sensitive to all three vertices. Users demand instant startup (low latency). Complex games and AR apps demand high frame rates (high throughput). And unlike servers, mobile devices have limited RAM and aggressive background process killing (low memory footprint). The history of mobile operating systems is largely a history of navigating this triangle.

1.3 Intermediate Representations (IR)

The "lingua franca" of modern compilation is the Intermediate Representation (IR). Both JIT and AOT compilers rarely translate high-level source code directly to machine code in one step. Instead, they lower source code to an IR, which abstracts away the specific hardware details while retaining enough structural information for optimization.

Java Bytecode / Dalvik Bytecode: In the Android ecosystem, Java or Kotlin source code is compiled into Java Bytecode (.class files). Traditionally, this stack-based bytecode was

converted into Dalvik Bytecode (.dex), which is register-based.[10] This conversion is crucial for mobile efficiency. A register-based architecture requires fewer instructions to perform the same operation compared to a stack-based architecture (which requires constant pushing and popping of operands), resulting in a smaller code size and fewer instruction dispatches, which saves battery.[11] The register-based design of Dalvik aligns more closely with the underlying ARM architecture, which is a load-store, register-heavy architecture.

LLVM IR: In the iOS ecosystem, and increasingly in Android's native components, LLVM IR plays a pivotal role. It is a Static Single Assignment (SSA) based representation that preserves type information and control flow graphs.[12] It allows for powerful optimization passes (dead code elimination, vectorization) to be decoupled from the frontend language (Swift, Objective-C, C++) and the backend architecture (ARM64). Apple's "Bitcode" concept was essentially a serialized form of LLVM IR uploaded to the App Store, allowing Apple to re-optimize the backend generation without the developer recompiling.[12] The ability to manipulate LLVM IR allows for optimizations like "App Thinning," where unnecessary architectures are stripped away before the user downloads the app.

---

2. Mobile-Specific Constraints

While desktop and server environments prioritize raw throughput, mobile environments operate under a strict set of physical and security constraints that fundamentally alter the calculus of runtime design.

2.1 The Energy Budget and Battery Physics

The most defining constraint of mobile computing is the finite energy budget. Every CPU cycle consumes energy. The relationship is roughly proportional to $C \times V^2 \times f$, where C is capacitance, V is voltage, and f is frequency.

JIT Energy Cost: JIT compilation introduces a "double tax" on energy. First, the CPU must expend energy to *compile* the code (running the JIT compiler algorithms). Second, it expends energy to *execute* the code. If a user runs an app for only a few seconds, the energy spent compiling complex methods may never be "paid back" by the efficiency gains of the generated native code.[2] For short-lived sessions, interpretation or AOT is strictly more energy-efficient. Studies have shown that JIT compilation overhead can increase memory usage by 18-50% and introduce significant startup latency, both of which correlate with higher energy consumption.[2]

The "Race to Idle": Mobile processors are heavily optimized for a "Race to Idle" strategy. The CPU is most efficient when it completes a task quickly and returns to a low-power sleep state (C-state). AOT supports this strategy better for intermittent tasks (like launching an app or checking a notification) because it executes immediately at peak speed. JIT, by contrast, keeps the processor active longer for the same task due to the compilation phase and the initial slower interpretation phase, draining more battery.[14] Quantitative benchmarks indicate that for simple operations, optimized JIT implementations can eventually achieve massive

energy reductions (up to 99.9%) over long durations due to superior code generation, but for the bursty nature of mobile usage, the initialization penalty often outweighs these long-term gains.[2]

2.2 The Security Architecture: $W \oplus X$

Modern mobile operating systems enforce a strict security principle known as **Write XOR Execute** ($W \oplus X$). This principle states that a page of memory can be either Writable (W) or Executable (X), but never both simultaneously.[16] This is a defense against code-injection attacks, such as buffer overflows, where an attacker writes shellcode to the stack or heap and then attempts to execute it.

This presents a fundamental existential challenge for JIT compilers. A JIT compiler functions by generating machine code (which requires writing to memory) and then jumping to that code (which requires executing memory).

- **The Conflict:** To generate code, the JIT needs PROT_WRITE. To run it, it needs PROT_EXEC.
- **The Workaround:** JIT engines typically allocate a page as Writable, write the machine code, and then use a system call (like mprotect on POSIX systems) to flip the permissions to Executable. Alternatively, they map the same physical memory twice: once as R/W and once as R/X (dual mapping).[17]

OS Restrictions:

- **iOS:** Apple's kernel strictly enforces $W \oplus X$ and forbids the "permission flipping" or "dual mapping" technique for standard third-party applications. This is why true JIT compilation is impossible for regular iOS apps (e.g., a Python interpreter on iOS cannot JIT compile). The dynamic-codesigning entitlement, which allows JIT, is reserved for system apps like the Safari web browser (JavaScriptCore).[18] Safari's JIT operates in a separate process that has this entitlement, heavily sandboxed to prevent exploitation.
- **Android:** Android is historically more permissive, allowing JIT compilation for user applications. However, newer versions of Android utilize hardened security features that monitor these transitions, and the system relies on the SELinux policies to contain potential JIT-spraying attacks where an attacker fills the JIT heap with shellcode.[16]

2.3 Storage vs. RAM

Mobile devices historically had limited NAND flash storage and restricted RAM compared to desktop peers.

- **AOT Impact:** AOT binaries are large. A compiled native executable is significantly larger than its bytecode equivalent (often 2x-3x larger). In the early days of Android (Lollipop), switching to pure AOT caused apps to consume massive amounts of disk space, leading to "Insufficient Storage" errors on budget devices.[20]
- **JIT Impact:** JIT saves disk space (storing only compact bytecode) but consumes RAM. The JIT compiler needs memory for its own internal data structures (control flow graphs, dependency trees) and the code cache. On a device with 1GB of RAM, a heavy JIT compiler (like V8 or the early JVM) can trigger frequent garbage collection or

low-memory kills.[9] The JIT compiler must essentially "steal" resources from the application it is trying to optimize.[14]

---

3. Case Study: The Evolution of Android (ART vs. Dalvik)

Android's history provides a perfect laboratory for observing the JIT vs. AOT trade-off in action. The platform has migrated from pure JIT to pure AOT, and finally to a sophisticated hybrid model, driven by the changing hardware landscape and user expectations.

3.1 Dalvik (Android 1.0 - 4.4): Trace-Based JIT

The original Android runtime, Dalvik, was designed for devices with extreme constraints (e.g., the T-Mobile G1 with 192MB RAM). Dalvik used a **Just-In-Time** compiler introduced in Android 2.2 (Froyo).[10]

- **Architecture:** It was a register-based VM, which is more compact than the stack-based JVM. This design choice was explicitly made to reduce instruction count and memory bandwidth pressure.
- **Trace JIT:** Unlike "Method JITs" which compile whole functions (like HotSpot), Dalvik used a "Trace JIT." It identified frequently executed *paths* (traces) through the code—even if they jumped across method boundaries—and compiled those linear traces.[21] This reduced memory usage because only the active paths were compiled, but it limited optimization scope compared to full method compilation.
- **Limitations:** As apps grew complex, the overhead of JIT and the stutter caused by compilation during UI interactions (jank) became unacceptable. The garbage collection mechanism in Dalvik was also stop-the-world and non-compacting, leading to fragmentation and further performance issues.

3.2 Early ART (Android 5.0 - 6.0): Pure AOT

In Android 5.0 (Lollipop), Google replaced Dalvik with the Android Runtime (ART). The initial version of ART employed a **Pure AOT** strategy.[21]

- **The Mechanism:** At installation time, a tool called dex2oat would take the app's DEX bytecode and compile it entirely into native machine code (.oat files). This .oat file is essentially an ELF binary containing the compiled code.
- **Benefits:** Apps launched instantly and ran smoothly with no JIT overhead. Battery life improved during execution because the CPU didn't have to spend cycles compiling code.
- **The Failure Mode:** The "Optimizing app..." screen during OS updates could take 30+ minutes as every installed app was recompiled. Install times for large apps (like Facebook) became agonizingly slow. Furthermore, the compiled binaries were massive, consuming gigabytes of storage on user devices.[20] This approach proved unsustainable as app sizes grew.

3.3 Modern ART (Android 7.0+): Profile-Guided Hybrid Model

To resolve the conflict, Android 7.0 (Nougat) introduced the current hybrid architecture, which is one of the most sophisticated runtimes in existence.[22] It balances the Iron Triangle by using all three execution modes: Interpretation, JIT, and AOT.

3.3.1 The Hybrid Lifecycle

1. **Installation:** The app installs instantly. No compilation is performed. The code remains as DEX bytecode.
2. **Initial Launch (Interpretation):** When the user opens the app, it runs in the **Interpreter**. This ensures zero startup latency (no waiting for JIT warmup).[22]
3. **Hot Path Detection (JIT):** As the app runs, the runtime profiles execution. Frequently used methods are flagged and handed to the **JIT compiler**. This provides high performance for the user's current session. The JIT generates code that is optimized for the current run but is not persisted to disk in a permanent way.[22]
4. **Profile Generation:** The runtime saves a "Profile" file that logs which methods were "hot" and which classes were loaded at startup. This profile captures the user's specific usage pattern of the app.
5. **Idle Compilation (AOT):** When the device is **idle and charging** (the perfect time for energy-intensive tasks), a background daemon (dex2oat) wakes up. It reads the Profile file and compiles *only* the hot methods identified in the profile into native code.[24] This is known as Profile-Guided Optimization (PGO).
6. **Subsequent Launches:** The next time the user launches the app, ART loads the pre-compiled AOT code for the hot paths (instant speed) and interprets the rest. This minimizes RAM usage (by not compiling rarely used code) while maximizing performance for common tasks.

3.3.2 Cloud Profiles (PGO)

Google took this a step further with "Cloud Profiles" (Android 9+).

- **Problem:** The first user of an app has to wait for local profiling to happen before they get AOT benefits.
- **Solution:** When millions of users run an app (e.g., YouTube), their devices generate similar profiles. Google Play aggregates these profiles in the cloud to create a "Core-Aggregated Code Profile".[25]
- **Mechanism:** When a new user downloads YouTube, they download the APK *and* this pre-computed profile (metadata). ART uses this profile to perform AOT compilation *during the download/install phase*.
- **Result:** The user gets AOT performance from the very first launch, without the massive storage penalty of compiling the *entire* app, because only the globally "hot" paths are compiled.[25]

3.3.3 Compiler Filters

The dex2oat compiler accepts various "filters" that dictate how much compilation occurs, allowing the system to balance storage and performance dynamically [24]:

- **verify**: Only runs DEX verification. No compilation. Used for fast installs.
- **quicken**: (Legacy) Optimizes some DEX instructions for better interpreter performance but doesn't generate native code.
- **speed-profile**: The default for most apps. Compiles only the methods listed in the profile.
- speed: Aggressively compiles all methods. Used for system apps or when "Force Compilation" is requested.
  This granular control allows Android to adapt to low-storage conditions by downgrading the filter (e.g., clearing AOT cache) without breaking the app.

---

4. Case Study: iOS & The "Walled Garden" Approach

Apple's approach to the runtime environment is diametrically opposed to Android's flexibility, driven by a philosophy of strict control, security, and hardware-software vertical integration.

4.1 Mandatory AOT and Security

iOS strictly prohibits JIT compilation for consumer applications. This is enforced via the kernel's refusal to grant the CS_OPS_STATUS or dynamic-codesigning entitlements to standard processes.[26]

- **Security Rationale:** If an app can generate executable code at runtime, it can theoretically download a script from a remote server, compile it, and execute it. This would bypass the App Store review process, allowing developers to change app behavior remotely (or malicious actors to inject code). By forcing AOT, Apple ensures that the code running on the device is exactly the code that was signed and reviewed.[18] The kernel checks the code signature of every page before allowing execution.
- **Performance Implication:** All iOS apps are compiled to native ARM64 machine code on the developer's machine (using Xcode/LLVM). This guarantees high performance and low startup time but results in larger download sizes.

4.2 The Exception: JavaScriptCore

The only exception to the "No JIT" rule is the **JavaScriptCore** engine used by Safari (and exposed via WKWebView).

- **Privileged JIT:** Safari runs in a multi-process architecture. The rendering and JavaScript execution happen in a separate "WebContent" process which *does* hold the dynamic-codesigning entitlement.[27]
- **JIT Region:** This process uses a special memory mapping (using the MAP_JIT flag in mmap) that the kernel recognizes. The kernel allows this specific region to be toggleable between Writable and Executable, but heavily guards it to prevent exploitation.[18] Third-party browsers (Chrome on iOS) are forced to use the slower

WebKit engine and cannot bring their own JITs (like V8), effectively capping their performance relative to Safari.

4.3 Bitcode and App Thinning

To manage the storage bloat of AOT binaries, Apple introduced **Bitcode** and **App Thinning** (specifically **App Slicing**).[29]

4.3.1 Bitcode (LLVM IR)

Until Xcode 14, developers could upload the **LLVM Intermediate Representation (Bitcode)** of their app to the App Store instead of the final machine code.[13]

- **Mechanism:** Apple's servers essentially acted as the final stage of the compiler. They would take the Bitcode and compile it into machine code for the specific device downloading the app.
- **Benefit:** If Apple released a new processor (e.g., moving from armv7 to arm64, or introducing a new instruction set extension), they could re-compile the app on their servers to take advantage of the new hardware without the developer needing to submit a new version.[12]

4.3.2 App Slicing

Because iOS binaries are AOT, a "Universal Binary" (Fat Binary) containing code for all architectures (armv7, arm64) and assets for all screen densities (@2x, @3x) is huge.

- **Slicing:** When a user with an iPhone 15 Pro (arm64, @3x OLED) downloads an app, the App Store "slices" the universal package. It delivers *only* the arm64 binary and the @3x assets. This drastically reduces the download size and storage footprint, mitigating the primary disadvantage of AOT.[29]

4.3.3 The Deprecation of Bitcode

With Xcode 14 (2022) and Xcode 15, Apple deprecated Bitcode.[12]

- **Analysis:** This move surprised many. It likely signals that the ARM64 architecture has stabilized enough that Apple no longer foresees a need to mass-migrate apps to a new architecture (like RISC-V) server-side in the near future. Additionally, maintaining binary compatibility with evolving LLVM IR versions is technically difficult.[32] The overhead of maintaining the Bitcode infrastructure likely outweighed the benefits, especially given that App Slicing solves the size issue without needing Bitcode re-compilation.

---

5. Cross-Platform Framework Theory (Flutter/React Native)

Cross-platform frameworks must navigate the native constraints of both Android and iOS while trying to offer a unified development experience. These frameworks have adopted vastly

different architectural approaches to the JIT/AOT dilemma.

5.1 Flutter: The Dual Mode Approach

Flutter, built on the Dart language, employs a unique strategy that explicitly switches compilation modes based on the development lifecycle.[33]

Mode	Compilation Strategy	Purpose
Debug Mode	**JIT (Just-In-Time)**	Enables **Hot Reload**. The Dart VM runs on the device, streaming source code updates. It compiles changes instantly, preserving the app's state. This provides a "web-like" dev cycle. The JIT allows for patching the running code without losing the current variable states.
Release Mode	**AOT (Ahead-Of-Time)**	For deployment, the Dart compiler performs "tree shaking" (dead code elimination) and compiles the entire codebase into a single native library (.so on Android, Dylib on iOS).

- **Architectural Insight:** Flutter carries its own rendering engine (Skia/Impeller) and does not use the OEM widgets. By compiling to AOT native code in release mode, Flutter bypasses the JavaScript bridge entirely, communicating directly with the canvas. This results in near-native performance (60/120 FPS) and fast startup, as there is no interpreter initialization.[34] The AOT compilation also helps in hiding the implementation details, offering better IP protection compared to shipping source code.

5.2 React Native: The Bridge vs. JSI

React Native has historically struggled with the performance cost of its runtime architecture, which relies on JavaScript (interpreted or JIT'd) controlling Native UI components.

5.2.1 The Legacy Architecture: The Bridge

- **Mechanism:** The JavaScript thread (running logic) and the Main/UI thread (rendering views) were completely separate. They communicated via an asynchronous JSON bridge.
- **Bottleneck:** Every time the JS code wanted to update a view, it had to serialize the command to JSON, send it over the bridge, and the native side had to deserialize it. This caused performance issues during rapid events (e.g., scroll animations), leading to the "white blank screen" where the native scroll was faster than the JS bridge could provide

content.[36] The serialization cost became the primary inhibitor of "native-like" feel.

5.2.2 The New Architecture: JSI (JavaScript Interface) & Fabric

To solve this, React Native introduced the **New Architecture** centered on **JSI**.[38]

- **JSI (JavaScript Interface):** Instead of serializing messages, JSI allows JavaScript to hold a reference to a C++ Host Object. It exposes native methods directly to the JS runtime.
- **Synchronous Execution:** JS can now call a method on a C++ object synchronously. This eliminates the serialization overhead entirely.
- **Fabric:** The new rendering system uses JSI to control the UI. When JS updates a component, it directly manipulates the C++ Shadow Tree, which the native OS renders. This allows React Native to prioritize UI updates and handle interactions without the "bridge lag".[40]
- **TurboModules:** This replaces the old Native Modules. TurboModules are lazy-loaded (initialized only when needed) and accessed via JSI, speeding up app startup significantly.[41]

5.2.3 Hermes: Bytecode Pre-compilation

Facebook developed the **Hermes** engine specifically for React Native on mobile.[42]

- **Design:** Unlike V8 (which is a JIT), Hermes is an **AOT-capable JavaScript engine**.
- **Bytecode AOT:** During the app build process, the JavaScript is parsed and compiled into Hermes Bytecode. The app ships with this bytecode.
- **Benefit:** The engine doesn't need to parse JS source or run a heavy JIT compiler at startup. It simply maps the bytecode into memory and executes. This drastically reduces TTI (Time to Interactive) and memory usage, aligning JS execution with the mobile "Iron Triangle" constraints.[43] This effectively gives React Native the startup benefits of AOT while keeping the flexibility of JS.

---

6. Advanced Optimization Concepts

To achieve high performance even within JIT/Interpretation constraints, mobile runtimes employ sophisticated optimization techniques. These mechanisms allow JIT compilers to often outperform static AOT compilers in peak throughput for specific workloads.

6.1 Inline Caching (IC)

Dynamic languages (JavaScript, Python, Smalltalk) suffer from slow property access. In the expression obj.x, the VM doesn't know where x is in memory because the shape of obj can change at runtime.

- **Mechanism:** Inline Caching caches the result of the lookup *at the call site*.
    - **Monomorphic:** If obj is always of type ClassA, the IC records the offset of x for ClassA. The generated machine code becomes a simple check if (type == ClassA)

return memory[offset]. This is extremely fast (1-2 machine instructions).[44]
    - **Polymorphic:** If obj alternates between ClassA and ClassB, the IC builds a small list (dispatch table) of shapes.
    - **Megamorphic:** If the site sees many different types (e.g., >4), the IC "gives up" and reverts to a generic (slow) hash table lookup to prevent code bloat.[46]
- **Mobile Relevance:** V8 (in Chrome/React Native) and the Dart VM rely heavily on ICs to make dynamic property access fast enough for 60fps animations. Maintaining "monomorphism" (using consistent types) is a key performance tip for mobile JS developers.[47]

6.2 On-Stack Replacement (OSR)

A common problem in JITs is a loop that runs for a long time inside a method that hasn't finished yet. If the JIT only compiles methods when they *enter*, this "hot loop" would run in the interpreter forever.

- **Mechanism:** OSR allows the runtime to switch from the interpreter to compiled code *in the middle of execution*. It pauses the interpreter, extracts the values of variables from the interpreter's stack frame, maps them to the CPU registers and stack layout of the newly compiled machine code, and jumps to the corresponding point in the native code.[48]
- **Reverse OSR (Deoptimization):** Conversely, if a speculative assumption fails (e.g., a variable assumed to be int suddenly becomes double), the native code performs **Deoptimization**. It reconstructs the interpreter's stack frame from its optimized state and jumps back into the interpreter. This safety net allows the JIT to be incredibly aggressive with optimizations.[4]

6.3 Speculative Optimization & The "Trap"

JIT compilers generate code based on the assumption that "the future will look like the recent past."

- **Type Specialization:** If a function add(a, b) is called with integers 1000 times, the JIT generates machine code for integer addition (which is fast).
- **Guards:** It inserts a "Guard" instruction before the addition: if (not_integer(a)) goto_deopt.
- **Cost:** If the application behavior changes phase (e.g., suddenly processing floats), these guards trigger deoptimization storms, where the CPU spends more time switching between JIT and Interpreter than doing work. Modern mobile runtimes (ART, V8) have "bailout" counters; if a method deoptimizes too often, the JIT marks it as "do not optimize" to stabilize performance.[50] This "bailout" prevents the pathological case where the JIT thrashes, consuming battery without making progress.

---

7. Performance and Energy Analysis

7.1 Startup vs. Throughput Data

Empirical studies quantify the trade-offs involved in selecting an execution model:

- **Startup:** AOT implementations (like GraalVM Native Image or ART AOT) consistently show startup times **40-70% faster** than JIT counterparts because they avoid the initialization overhead of the JIT compiler and the initial interpretation phase.[2] This is critical for mobile apps where user abandonment rates increase significantly with every second of delay.
- **Throughput:** For long-running, complex numerical tasks (e.g., Mandelbrot sets, matrix multiplication), JIT can outperform AOT by up to **20-50%** in specific scenarios. This is because JIT can utilize runtime profiling to perform optimizations that are unsafe for a static compiler, such as aggressive virtual call devirtualization and branch prediction based on real data. However, in mobile UI workloads (which are bursty and interrupt-driven), this advantage is often negated by the "warm-up" jank.[14]

7.2 Energy Implications

- **JIT Overhead:** Research indicates that the JIT compilation process itself can increase memory overhead by **18-50%** and consumes significant CPU cycles.[2] On battery-powered devices, this "compilation energy" is a parasitic loss.
- **The "Race to Idle":** Mobile processors are most efficient when they complete a task quickly and return to a low-power sleep state (Race to Idle). AOT supports this strategy better for short tasks (launching an app, checking a notification) because it executes immediately at peak speed. JIT keeps the processor active longer for the same task due to the compilation phase, draining more battery.[14] The energy cost of JIT is essentially an investment; it only pays off if the application runs long enough for the optimized code's efficiency to recoup the initial compilation cost. For many mobile use cases (e.g., checking the time, replying to a text), this payback period is never reached.

---

8. Conclusion

The architectural dichotomy between JIT and AOT compilation is no longer a binary choice but a spectrum of strategic compromises tailored to the unique constraints of mobile hardware.

1. **Convergence to Hybrid Models:** The industry is converging on hybrid solutions. Android's evolution from Dalvik (Pure JIT) to Early ART (Pure AOT) and finally to Modern ART (Profile-Guided Hybrid) demonstrates that neither extreme is ideal. The Hybrid model captures the low-storage/fast-install benefits of JIT and the high-performance/low-energy benefits of AOT.
2. **Security Dictates Architecture:** In the iOS ecosystem, the security model (specifically the restriction on dynamic code generation) effectively mandates AOT. This constraint has forced Apple to invest heavily in LLVM compiler technologies (Bitcode, App Slicing)

to mitigate the inherent downsides of AOT (binary size).

3. **The Rise of "Cloud Compilation":** The introduction of Android Cloud Profiles and the (now deprecated) iOS Bitcode suggests a trend where the "Compilation" vertex of the Iron Triangle is offloaded to the cloud. By aggregating usage data from millions of devices, app stores can deliver pre-optimized binaries that offer JIT-like profiling intelligence with AOT-like startup performance.

For students of mobile systems, understanding these mechanisms—from the battery cost of a JIT cycle to the security implications of a Writable+Executable memory page—is essential. The future of mobile runtimes lies not in choosing JIT *or* AOT, but in orchestrating them seamlessly to hide the cost of abstraction from the user, delivering applications that are instantly responsive, battery-efficient, and secure.

Works cited

1. JIT vs AOT Compilation: Understanding the Two Pillars of Modern Software Performance, accessed December 11, 2025, https://medium.com/@ayasc/jit-vs-aot-compilation-understanding-the-two-pillars-of-modern-software-performance-7fda843f8f77
2. (PDF) Ahead-of-Time vs. Just-in-Time Compilation Trade-offs ..., accessed December 11, 2025, https://www.researchgate.net/publication/396770709_Ahead-of-Time_vs_Just-in-Time_Compilation_Trade-offs_Empirical_performance_studies
3. Just-in-time compilation - Wikipedia, accessed December 11, 2025, https://en.wikipedia.org/wiki/Just-in-time_compilation
4. Speculative Optimization in the JVM | by Ondrej Kvasnovsky - Medium, accessed December 11, 2025, https://ondrej-kvasnovsky.medium.com/speculative-optimization-in-the-jvm-understanding-runtime-performance-e1fbb7061426
5. Correctness of Speculative Optimizations with Dynamic Deoptimization - Gallium, accessed December 11, 2025, https://gallium.inria.fr/~scherer/research/sourir/sourir.pdf
6. Elixir Can Save Your Company Money - DockYard, accessed December 11, 2025, https://dockyard.com/blog/2023/01/30/elixir-can-save-your-company-money
7. Embedded Development - Timesys, accessed December 11, 2025, https://www.timesys.com/blog/embedded-development/
8. AOT vs. JIT: Impact of Profile Data on Code Quality - Institute for Information Sciences, accessed December 11, 2025, https://www.ittc.ku.edu/~kulkarni/CARS/papers/pgo.pdf
9. Comparing AOT and JIT Compilers: Understanding the Differences and Making an Informed Choice — Programming concepts - Amin Nezampour, accessed December 11, 2025, https://aminnez.com/programming-concepts/jit-vs-aot-compiler-pros-cons
10. Dalvik (software), accessed December 11, 2025, https://grokipedia.com/page/Dalvik_(software)
11. Swift: A Register-based JIT Compiler for Embedded JVMs - Yuan Zhang,

accessed December 11, 2025, https://yuanxzhang.github.io/paper/swift-vee2012.pdf
12. ios - Xcode 14 deprecates bitcode - but why? - Stack Overflow, accessed December 11, 2025, https://stackoverflow.com/questions/72543728/xcode-14-deprecates-bitcode-but-why
13. App Thinning in iOS: Bitcode, Slicing & On-Demand Resources | by Tanishq Arora - Medium, accessed December 11, 2025, https://medium.com/@ios-interview/app-thinning-in-ios-bitcode-slicing-on-demand-resources-b8349613af39
14. Why is Android Runtime's AOT compilation more performant than Dalvik's JIT? [closed], accessed December 11, 2025, https://softwareengineering.stackexchange.com/questions/274640/why-is-android-runtimes-aot-compilation-more-performant-than-dalviks-jit
15. Hybrid Execution: Combining Ahead-of-Time and Just-in-Time Compilation | Request PDF - ResearchGate, accessed December 11, 2025, https://www.researchgate.net/publication/374856306_Hybrid_Execution_Combining_Ahead-of-Time_and_Just-in-Time_Compilation
16. Protecting Bare-metal Embedded Systems With Privilege Overlays - GitHub Pages, accessed December 11, 2025, https://helix979.github.io/jkoo/pdf/[Oakland17]%20Protecting%20Bare-metal%20Embedded%20Systems%20With%20Privilege%20Overlays.pdf
17. Guide to implementing Forths on modern systems with W ^ X locks. - Reddit, accessed December 11, 2025, https://www.reddit.com/r/Forth/comments/1iripfe/guide_to_implementing_forths_on_modern_systems/
18. Computer Science 161 Summer 2019 Dutra and Jawale, accessed December 11, 2025, https://inst.eecs.berkeley.edu/~cs161/su19/lectures/lec04_defenses.pdf
19. Jailed Just-in-Time Compilation on iOS - Saagar Jha, accessed December 11, 2025, https://saagarjha.com/blog/2020/02/23/jailed-just-in-time-compilation-on-ios/
20. Dalvik, ART, JIT, and AOT in Android - Outcome School, accessed December 11, 2025, https://outcomeschool.com/blog/dalvik-art-jit-aot
21. Android Runtime - Wikipedia, accessed December 11, 2025, https://en.wikipedia.org/wiki/Android_Runtime
22. Implement ART just-in-time compiler - Android Open Source Project, accessed December 11, 2025, https://source.android.com/docs/core/runtime/jit-compiler
23. Why does Android use JIT and AOT? : r/androiddev - Reddit, accessed December 11, 2025, https://www.reddit.com/r/androiddev/comments/1dtlym9/why_does_android_use_jit_and_aot/
24. Configure ART | Android Open Source Project, accessed December 11, 2025, https://source.android.com/docs/core/runtime/configure
25. Improving app performance with ART ... - Android Developers Blog, accessed December 11, 2025,

https://android-developers.googleblog.com/2019/04/improving-app-performance-with-art.html
26. dynamic-codesigning gates MAP_JIT, which is what MobileSafari uses for achieving... | Hacker News, accessed December 11, 2025, https://news.ycombinator.com/item?id=18429757
27. iOS 16 - restricted Userclients | ios16_restricted_iouserclients, accessed December 11, 2025, https://saaramar.github.io/ios16_restricted_iouserclients/
28. Allow execution of JIT-compiled code entitlement | Apple Developer Documentation, accessed December 11, 2025, https://developer.apple.com/documentation/bundleresources/entitlements/com.apple.security.cs.allow-jit
29. App size after using realm · Issue #2581 - GitHub, accessed December 11, 2025, https://github.com/realm/realm-cocoa/issues/2581
30. Top iOS developer interview questions and answers in 2025 - Turing, accessed December 11, 2025, https://www.turing.com/interview-questions/ios
31. App sizes are out of control - Hacker News, accessed December 11, 2025, https://news.ycombinator.com/item?id=14901051
32. Apple rejects apps with bitcode and WebRTC compiled with Chromium toolchain [42224164], accessed December 11, 2025, https://issues.webrtc.org/issues/42224164
33. Flutter JIT & AOT Explained: The Secret Behind Fast Development & Smooth Apps | by NizzCorp Academy | Medium, accessed December 11, 2025, https://medium.com/@nizzcorpacademy/flutter-jit-aot-explained-the-secret-behind-fast-development-smooth-apps-ccaa0646ba9c
34. Utilizing Dart's Native Compilation for Lightning-Fast Flutter Apps - Vibe Studio, accessed December 11, 2025, https://vibe-studio.ai/insights/utilizing-dart-s-native-compilation-for-lightning-fast-flutter-apps
35. Benchmarking Flutter vs. React Native: Performance Deep Dive 2025 - Vibe Studio, accessed December 11, 2025, https://vibe-studio.ai/insights/benchmarking-flutter-vs-react-native-performance-deep-dive-2025
36. Flutter vs. React Native: Your Ultimate Cross-Platform App Framework Showdown, accessed December 11, 2025, https://acubetechnologies.com/blogs/flutter-vs-react-native-your-ultimate-cross-platform-app-framework-showdown
37. New Architecture is here - React Native, accessed December 11, 2025, https://reactnative.dev/blog/2024/10/23/the-new-architecture-is-here
38. How does React Native's New Architecture affect performance? - DEV Community, accessed December 11, 2025, https://dev.to/amazonappdev/how-does-react-natives-new-architecture-affect-performance-1dkf
39. Deep Dive into React Native's New Architecture: JSI, TurboModules, Fabric & YogaSQL Databases in Fabric? | ESPC Conference, 2025, accessed December 11, 2025,

https://www.sharepointeurope.com/deep-dive-into-react-natives-new-architecture-jsi-turbomodules-fabric-yoga/
40. Fabric - React Native, accessed December 11, 2025, https://reactnative.dev/architecture/fabric-renderer
41. 0.73: Introducing Bridgeless Mode · reactwg react-native-new-architecture · Discussion #154 - GitHub, accessed December 11, 2025, https://github.com/reactwg/react-native-new-architecture/discussions/154
42. Garbage Collector not picking up high memory pressure when memory is allocated in native HostObjects · Issue #982 · facebook/hermes - GitHub, accessed December 11, 2025, https://github.com/facebook/hermes/issues/982
43. AIEnergy: An energy benchmark for AI-empowered mobile and IoT devices - ITU, accessed December 11, 2025, https://www.itu.int/dms_pub/itu-s/opb/jnl/S-JNL-VOL6.ISSUE2-2025-A15-PDF-E.pdf
44. Inline caching - Wikipedia, accessed December 11, 2025, https://en.wikipedia.org/wiki/Inline_caching
45. The V8 Engine Series III: Inline Caching — Unlocking JavaScript Performance, accessed December 11, 2025, https://braineanear.medium.com/the-v8-engine-series-iii-inline-caching-unlocking-javascript-performance-51cf09a64cc3
46. What's up with monomorphism?, accessed December 11, 2025, https://mrale.ph/blog/2015/01/11/whats-up-with-monomorphism.html
47. Understanding Monomorphism to Improve Your JS Performance up to 60x - Builder.io, accessed December 11, 2025, https://www.builder.io/blog/monomorphic-javascript
48. A Modular Approach to On-Stack Replacement in LLVM - Sable Research Group, accessed December 11, 2025, https://www.sable.mcgill.ca/publications/techreports/2012-1/lameed-12-osr-TR.pdf
49. runtime/docs/design/features/OnStackReplacement.md at main - GitHub, accessed December 11, 2025, https://github.com/dotnet/runtime/blob/main/docs/design/features/OnStackReplacement.md
50. Land ahoy: leaving the Sea of Nodes - V8 JavaScript engine, accessed December 11, 2025, https://v8.dev/blog/leaving-the-sea-of-nodes
51. Grokking V8 closures for fun (and profit?), accessed December 11, 2025, https://mrale.ph/blog/2012/09/23/grokking-v8-closures-for-fun.html